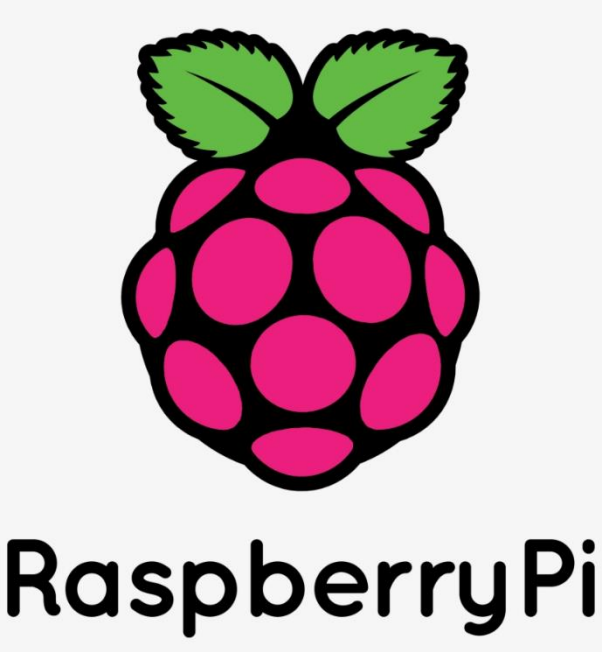
1TPIF

PIF DT 2022-2023

User story 5 Labo: Controller



PEREIRA MOREIRA Hugo (LN)

Inhaltsverzeichnis

Intro 2
Firmware 2
Exception happened 3
Door access granted 3
Door access denied 3
Should: Thermometer 4



Controller

Intro

Das Ziel dieser User Story war es, meinen Controller so zu installieren und zu konfigurieren, dass er auf unsere Website zugreifen und auch die Tür aufschließen kann, wenn der Benutzer eine Reservierung vorgenommen hat, andernfalls wird sie nicht geöffnet. Damit alles funktioniert, musste ich einige Änderungen am Python-Code vornehmen.

Firmware

Zuerst installierte ich alles, was ich brauchte, um das Thermometer, den Buzzer, die Leds und alles was auf der Platine habe, benutzen zu können. Als nächstes habe ich die erhaltene Firmware auf den Raspberry kopiert. Als erstes habe ich den Pfad meines Servers ins Firmware geschrieben, so dass die Firmware zum php vom Controller umgeleitet wird.

```
WEBSERVICE_ENDPOINT = "https://192.168.111.45/php/authorization/authorization_rooms.php"
HTTP_REQUEST_TIMEOUT_SECONDS = 5    # number of seconds before interrupting http request
```

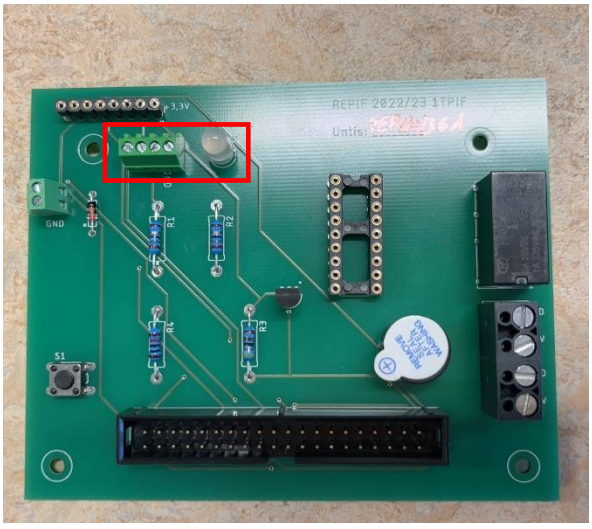
Nachdem ich das getan hatte, habe ich mein Pyhton-Skript geteste, es funktionierte aber nichts. Ich war meine Log-Dateien sehen und es gab mir einen https-Fehler, weil meine Seite ein SSL von mir gemacht hat, so musste ich ein bisschen suchen und ich fand einen Weg, der mir geholfen hat, so dass die Firmware die Warnnachrichten ignoriert, wenn sie die Website betritt.
Ich habe den „import urllib3“ ganz oben auf dem Code gesetzt

```
import urllib3
from w1thermsensor import W1ThermSensor
sensor = W1ThermSensor()
```

Weiter unten habe ich die folgende Zeile in den Code eingefügt, um auch Warnungen zu ignorieren

```
# this function sends a GET request to the webservice endpoint with optional http parameters
def getJSON( httpParameters={} ):
    payload = httpParameters
    print("Sending request to ", WEBSERVICE_ENDPOINT, " with rfid:\'", httpParameters['rfid'], "\' and hostname=\'", httpParameters['hostname'], "\'", sep='')
    urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
    response = requests.get(WEBSERVICE_ENDPOINT, params=payload, timeout=HTTP_REQUEST_TIMEOUT_SECONDS, verify=False)
    data = response.json()
    return data
```

Danach ging ich erneut zum Test und die Platine reagierte bereits. Es war hier, dass ein anderes Problem erschien, ich machte Änderungen im pyhton, um die Farbe der Leds zu ändern, aber es änderte nichts, sondern sie blieb immer blau. Der Grund war, weil wir zwei Leds an die Platine angeschlossen benötigen und wir hatten nur eine, die eine die ich gelötet habe.



Ich sprach mit meinem Lehrer und wir testeten den Anschluss einer anderen LED an diesen grünen Eingang und es funktionierte. Es hat die Farbe geändert. Und hier begann ich mit meinen Änderungen an Python, damit es alles tat, was ich machen müsste.

Exception happened

Ich habe noch nie Python programmiert und es war nicht allzu schwierig für mich. Ich werde damit beginnen, zu erklären, was ich geändert habe, wenn es einen Fehler in der Kommunikation zwischen der Raspberry und dem Server gibt.

```
# this function defines the routine when an exception occurs such as:
# webservice not reachable, http timeout, problem parsing json data ...
def runExceptionRoutine():
    print("Exception happened!")
    for i in range(0, 3):
        showColor((255, 0, 0)) # show green on first rgb led
        beep(1)
        time.sleep(0.5)
        showColor((0, 0, 0))  # turn first rgb off
        time.sleep(0.5)
```

Bei einem Fehler blinkt es dreimal rot und piept dreimal. Dazu habe ich eine Schleife von 0 bis 3 gemacht. In der Schleife schrieb ich, die Farbe auf Rot zu ändern, 1 Mal piepen, dann 0.5 Sekunden time_sleep, d.h 0.5 Sekunden soll er piepen und die Farbe zeigen, und dann die LED auszuschalten und für 0,5 Sekunden auszuschalten und dann soll er das noch 2 mal tun, weil die schleife von 0 bis 3 ist.

Door access granted

Wenn wir Zugang haben, um die Tür zu öffnen, muss die erste LED grün werden und einmal piepen. Die LED sollte eingeschaltet bleiben, bis die Tür schließt.

```
# this function defines the routine when permission has been granted to open the lock
def permissionGrantedRoutine():
    beep(1)
    showColor((0, 255, 0))
    print("door access granted")
    openLock()
    time.sleep(SOLENOID_TIMEOUT)
    closeLock()
    showColor((0, 0, 0))
```

Die Art und Weise, wie ich es tat, war, 1 Mal zu piepen und dann die LED auf Grün zu ändern. Dann soll „Door access granted“ ins Log geschrieben werden. Dann wird die Tür mit openLock() geöffnet. Nach dem time_sleep im SOLENOID_TIMEOUT, das in diesem Fall 5 Sekunden beträgt, wird die Tür wieder geschlossen mit closeLock() und schaltet schließlich die LED aus.

Door access denied

Wenn der Benutzer keine Berechtigung zum Öffnen der Tür hat, muss die LED zwei Sekunden lang rot bleiben und dann ausgehen und zwei Pieptöne abgeben.

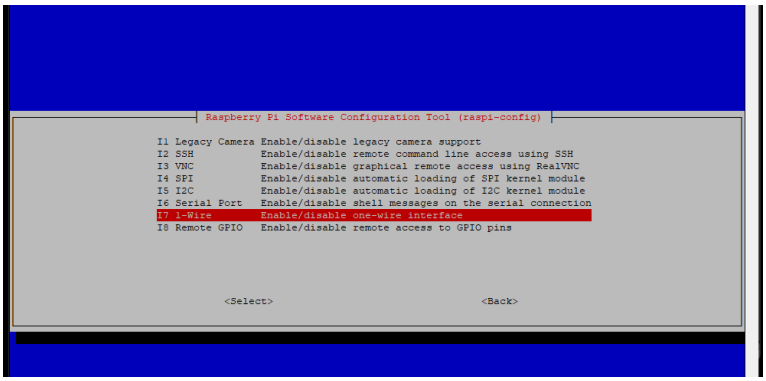
```
# this function defines the routine when permission has been denied to open the lock
def permissionDeniedRoutine():
    print("door access denied")
    showColor((255, 0, 0))
    beep(2)
    time.sleep(2)
    showColor((0, 0, 0,))
```

Dazu muss die Funktion die LED auf Rot schalten, dann beep(2), um zweimal zu piepen und dann time.sleep(2), um die LED nach zwei Sekunden wieder auszuschalten

Should: Thermometer

Die zweite RGB-Led zeigt den Zustand der Temperatur an, die mit Hilfe des Temperatursensors ausgelesen wird. Sie kann z.B. zwischen grün und rot wechseln je nachdem ob die aktuelle Temperatur einen Grenzwert übersteigt oder nicht.
Dazu musste ich das Paket installieren, damit das Thermometer ausgelesen werden konnte.

```
Processing triggers for desktop-file-utils (0.26-1) ...
perhu361@raspberrypi:~/firmware $ sudo pip3 install w1thermsensor
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting w1thermsensor
  Downloading https://www.piwheels.org/simple/w1thermsensor/w1thermsensor-2.0.0-py2.py3-none-any.whl (18 kB)
Requirement already satisfied: click in /usr/lib/python3/dist-packages (from w1thermsensor) (7.1.2)
Installing collected packages: w1thermsensor
Successfully installed w1thermsensor-2.0.0
```



Und dann habe ich das Thermometer ganz oben am Anfang von Python importiert. Ich habe auch einen variablen "Sensor" gemacht, der das Thermometer ist.

```
import requests, json, socket
import time, board, neopixel
import urllib3
from w1thermsensor import W1ThermSensor
sensor = W1ThermSensor()
```

Dann habe ich die Funktion von der ersten LED kopiert, um eine Funktion für die zweite LED zu machen. Ich habe nur den Namen der Funktion geändert und den pixel, damit wir später sagen können, was die zweite LED zu tun hat

```
# this function displays a color on the first rgb led
def showColor( color ):
    pixels[0] = color
    pixels.show()

# this function displays a color on the second rgb led
def showColor2( color ):
    pixels[1] = color
    pixels.show()

# this function opens the solenoid lock
```

Nachdem ich das alles erledigt hatte, ging ich zur Funktion über. Ich habe die Funktion in die Funktion eingefügt, die startet, sobald sich den Raspberry einschaltet. Zuerst habe ich eine Variable erstellt, die die Zeit der letzten Temperaturprüfung speichert. Dann habe ich eine if-Funktion erstellt, die einen Wert hat. Wenn die Zeit in diesem Moment - die Zeit der letzten Überprüfung größer als 10 Sekunden ist, soll den raspi die Temperatur messen diese in den Logs speichern und dann macht er die neue Zeit in der Variablen der letzten Überprüfung. Als nächstes haben wir eine weitere if-Funktion, die sagt, wenn die Temperatur größer als 25 ist, wechselt die zweite LED auf rot und und wenn sie kleiner als 25 ist, wechselt die zweite LED auf grün.

```
runStartupRoutine()
# set the initial last_temp_check_time to the current time
last_temp_check_time = time.time()
while True:
    #id, text = reader.read() # try to read an rfid batch. this is blocking!

    # we use the non-blocking read so we can still react on the temperature values
    id, text = reader.read_no_block() # try to read an rfid batch. this is not blocking!

    # check if 5 minutes have passed since the last temperature check
    if time.time() - last_temp_check_time >= 10:
        # get the temperature reading
        temperature = sensor.get_temperature()

        # print the temperature reading
        print("Temperature: ", temperature ,"°C")

        # update the last_temp_check_time to the current time
        last_temp_check_time = time.time()

        # check if temperature is higher than 25 degrees Celsius
        if temperature > 25:
            # change the LED color to red if above 25 degrees Celcius
            showColor2((255, 0, 0))
        else:
            # change the LED color to green if under 25 degrees Celcius
            showColor2((0, 255, 0))
```